Seguridad, Rendimiento y Disponibilidad de Servicios e Infraestructuras

Ana I. González Acuña

Grado en Ingeniería Informática



2026/27

SRDSI

1. Introducción a la seguridad informática
2. Seguridad en la comunicación
3. Seguridad en los servicios
4. Seguridad Operacional



Tema 2: Seguridad en la comunicación



2 Seguridad en la comunicación
2.1 Protocolos seguros de comunicación
2.2 Nivel de Aplicación
2.3 Nivel de Transporte
2.4 Nivel de Interred
2.5 Nivel de Enlace



2 Seguridad en la comunicación
2.1 Protocolos seguros de comunicación
2.2 Nivel de Aplicación
- SSH
- PGP
- S/MIME
2.3 Nivel de Transporte
2.4 Nivel de Interred
2.5 Nivel de Enlace

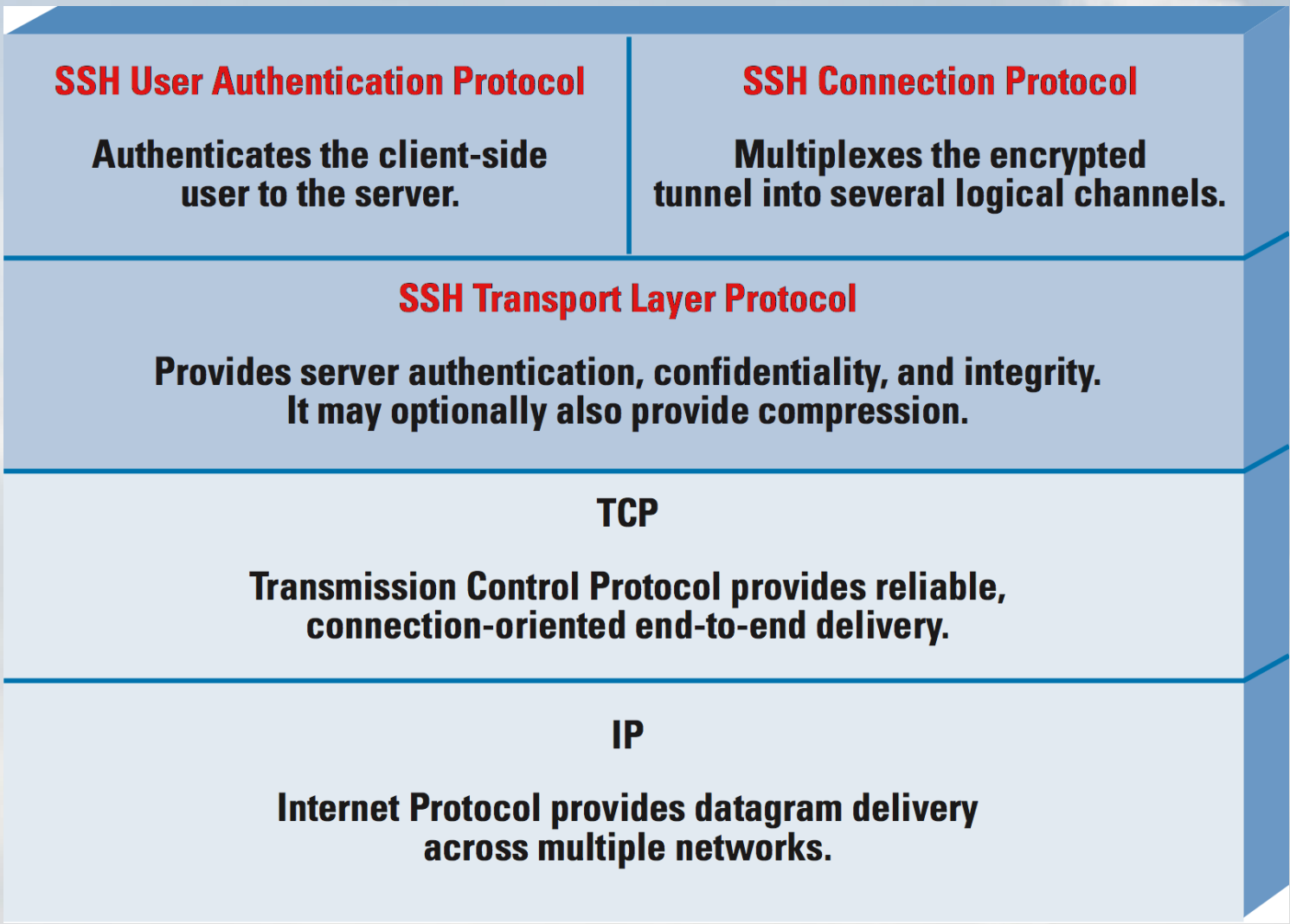


Secure **Sh**ell (RFC 4250-56, 9142)

- SSH es un **protocolo** del nivel de aplicación para crear **conexiones seguras** entre dos sistemas sobre redes no seguras.
- Modelo cliente/servidor (`ssh`/`sshd`)
- Alternativa a programas no seguros de acceso remoto como `telnet`, `rlogin` y `rsh`
- Base para protocolos de transferencia segura de archivos: `scp` y `sftp`
- Permite la creación de túneles seguros para otras aplicaciones
- Autenticación del servidor [y del usuario]. Integración en sistemas de autenticación como Kerberos, PGP, PAM ...
- Implementaciones: OpenSSH software libre (requiere OpenSSL), Tectia-ssh versión comercial y empresarial
- Librerías para usar SSH:libssh, wolfssh...

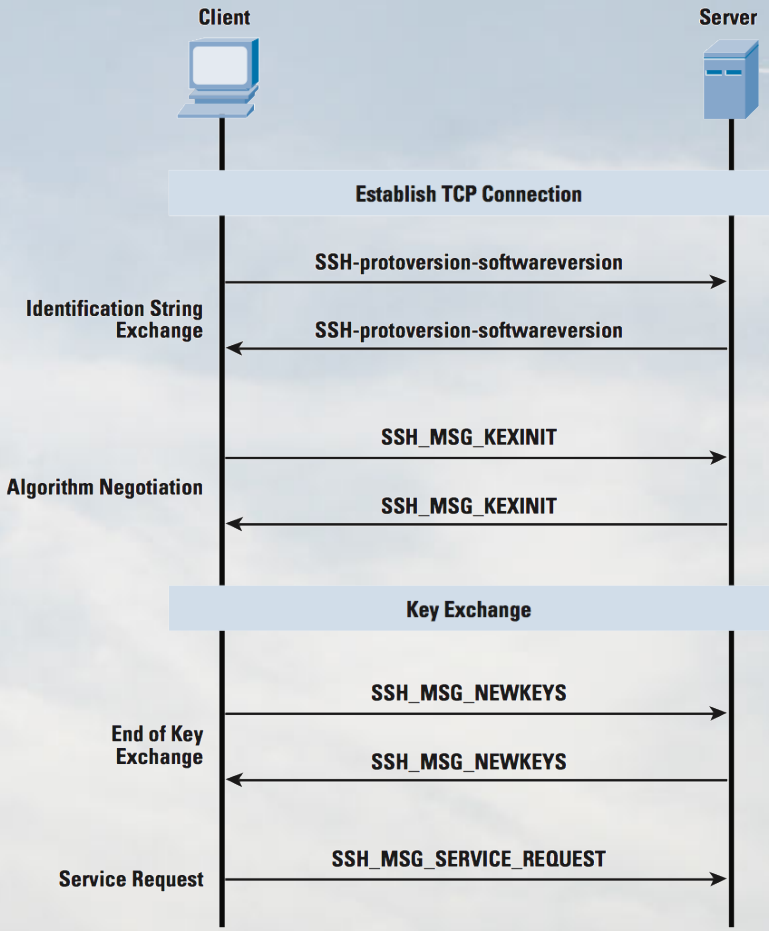


Arquitectura SSH (RFC 4251)



fuente: W. Stallings "SSH" IPJ-2009

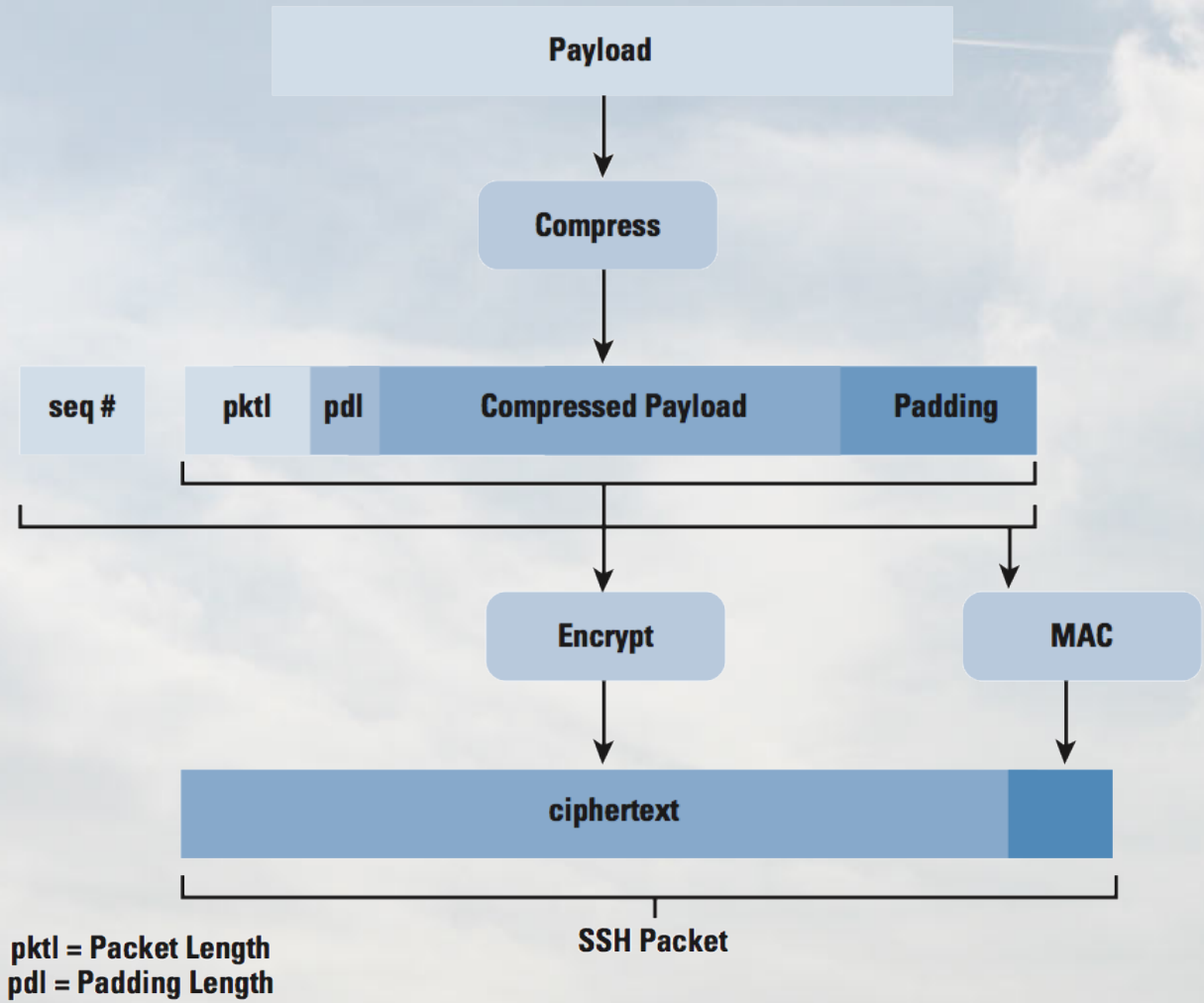
SSH Transport Layer Protocol (RFC 4253, 8308)



fuente: W. Stallings "SSH" IPJ-2009

RFC 8308, modifica el protocolo para permitir negociar extensiones tras la fase de intercambio de claves (`SSH_MSG_EXT_INFO` Message)

Formato paquetes SSH



fuente: W. Stallings "SSH" IPJ-2009

SSH User Authentication Protocol (RFC 4252)

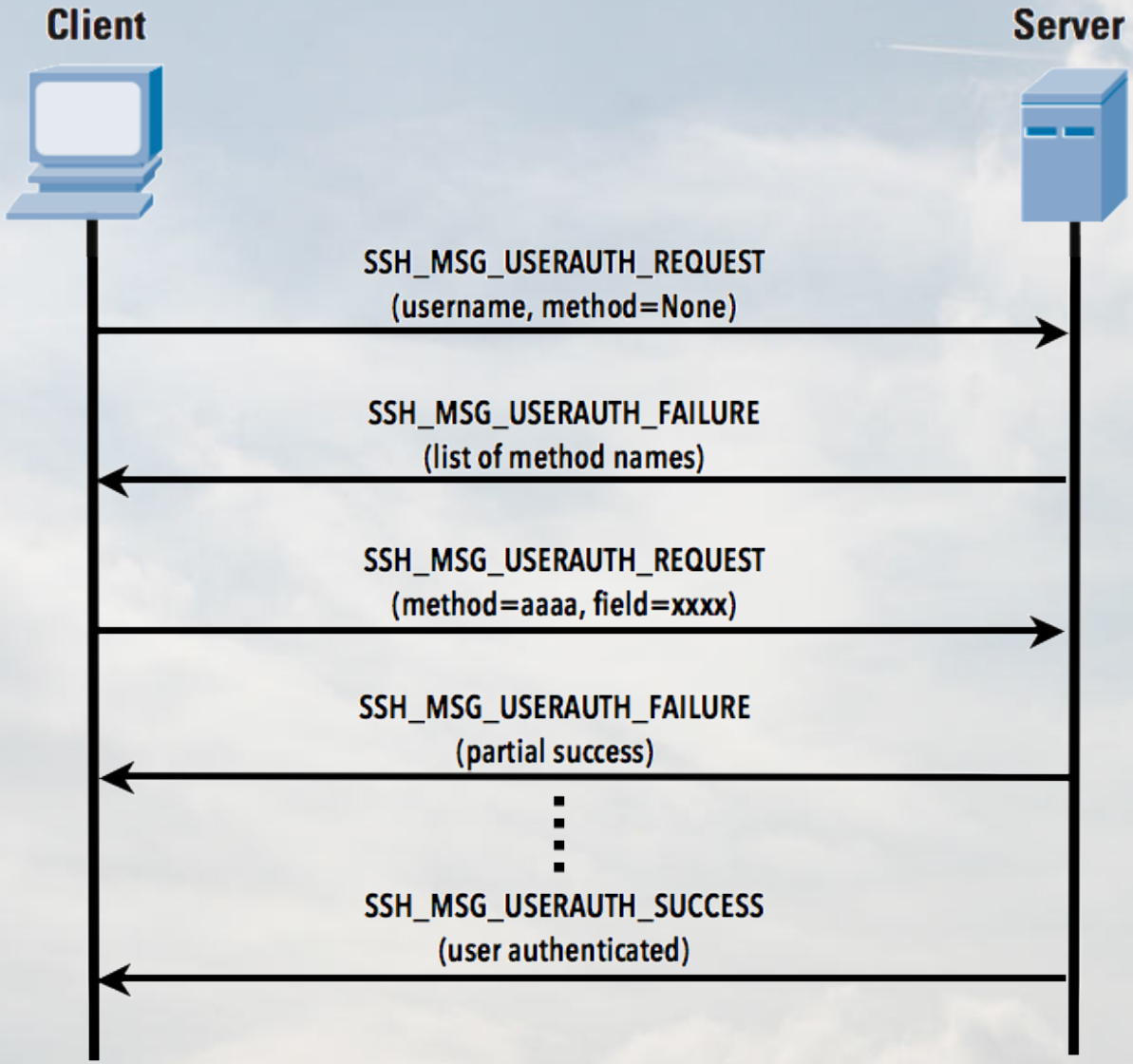
Métodos para autenticar al usuario ante el servidor:

- Autenticación nula: acceso directo
- Autenticación basada en contraseña
- Autenticación basada en clave pública [claves o certificados SSH]
- Autenticación basada en el host
- Autenticación realizada por otro sistema (PAM, Kerberos,...)

NIST-IR-7966



Proceso de autenticación

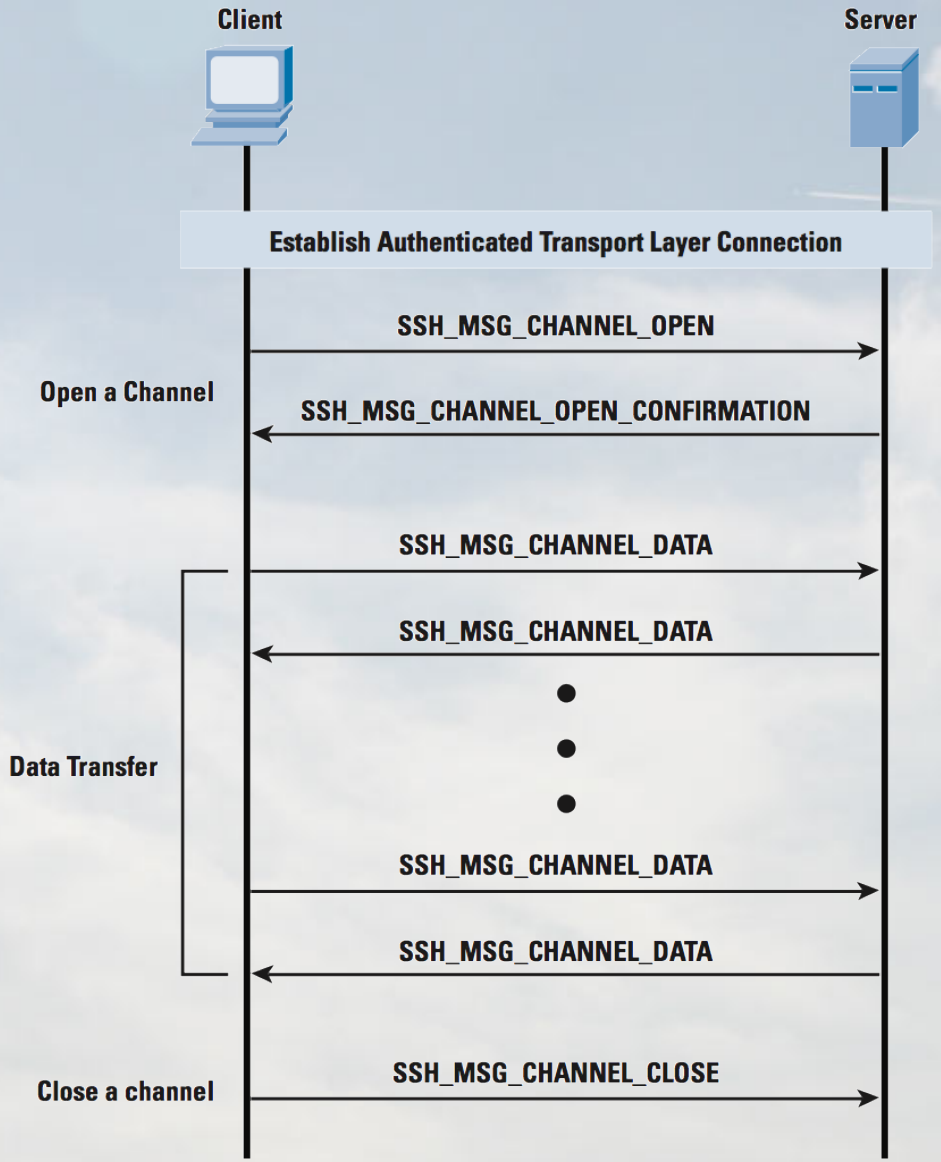


SSH Connection Protocol (RFC 4254)

Gestiona las conexiones establecidas. Se encarga del envío de los datos entre extremos.
En una conexión SSH (***tunnel***) se pueden multiplexar distintos **canales lógicos** de comunicación:

- Sesiones interactivas:
  - Ejecución remota de programas (*shell, file transfer, e-mail, system command*, ...)
  - Reenvío de conexiones X11: *remote GUI*
- Reenvío de conexiones TCP/IP:
  - *Local Port Forwarding (direct-tcpip)*: *SSH-tunnel* de cliente a servidor
  - *Remote Port Forwarding* (*forwarded-tcpip*): *SSH-tunnel* de servidor a cliente





fuente: W. Stallings "SSH" IPJ-2009

Configuración servidor OpenSSH

Fichero de configuración: /etc/ssh/sshd_config

```
Protocol 2                                  Solo permitir v2
Port 2222                                   NO usar el puerto estándar
ListenAddress 0.0.0.0                       Solo IPv4 (:: IPv6)
HostKey path-file-keys
PermitEmptyPasswords no
PermitRootLogin no
TCPKeepAlive yes
PubkeyAuthentication yes
MaxAuthTries 5
AuthorizedKeysFile .ssh/authorized_keys
AllowTcpForwarding no
AllowUsers <user1> <user2> ...              Restricción de usuarios y grupos
DenyGroups <group1> ...
```

sshd_config 

Configuración cliente SSH

Datos de configuración, obtenidos de las siguientes fuentes:

1. Opciones por línea de comandos
2. Fichero de configuración del usuario: `~/.ssh/config`
3. Fichero de configuración para el sistema: `/etc/ssh/ssh_config`

```
Protocol 2                          Solo permitir v2
Port 2222                           NO usar el puerto estándar
ServerAliveInterval 30
StrictHostKeyChecking yes           NO añadir automáticamente server-keys
Host labos                          Sección aplicable solo al host labos
    IdentityFile ~/.ssh/srdsi.pem   Fichero alternativo de autenticación
    Hostname the.srdsi.lab
    User yomismo
    Port 12345
    PasswordAuthentication yes
```



Acceso remoto seguro

- Conexión remota segura
  `ssh [-v[v[v]]] {[-l` *remote-user* `],[`*remote-user*`@]}`*remote-host*
- Ejecución remota segura de un comando
  `ssh {[-l` *remote-user* `],[`*remote-user*`@]}`*remote-host* *command*



Transferencia segura de archivos

- Copiar un fichero local a un sistema remoto
  `scp` *`localfile user@remote-`*`host:`*`filepath`*
- Copiar un fichero remoto al sistema local
  `scp` *`user@remote-host:filepath localfile`*
- Establecer una conexión interactiva y segura con SSH-FTP
  `sftp` *`user@remote-host`*



Autenticación del usuario mediante clave pública

- Generar par de claves asimétricas (desde `$HOME`)
  `ssh-keygen -t {rsa, ~~dsa~~, ecdsa, ed25519} -b {2048, 384, 256 }`
- Copiar la clave pública en otros *hosts*
  `ssh-copy-id [-i ~/.ssh/id_rsa]` *`user@servidor`*
- Gestión automática mediante agente de las claves privadas del usuario
  `eval $(ssh-agent)`
  `ssh-add`



Gestión de claves SSH

Autenticación basada en clave pública —> Gestión de claves SSH

- Generación de claves
- Uso
- Almacenamiento
- Recuperación
- Destrucción

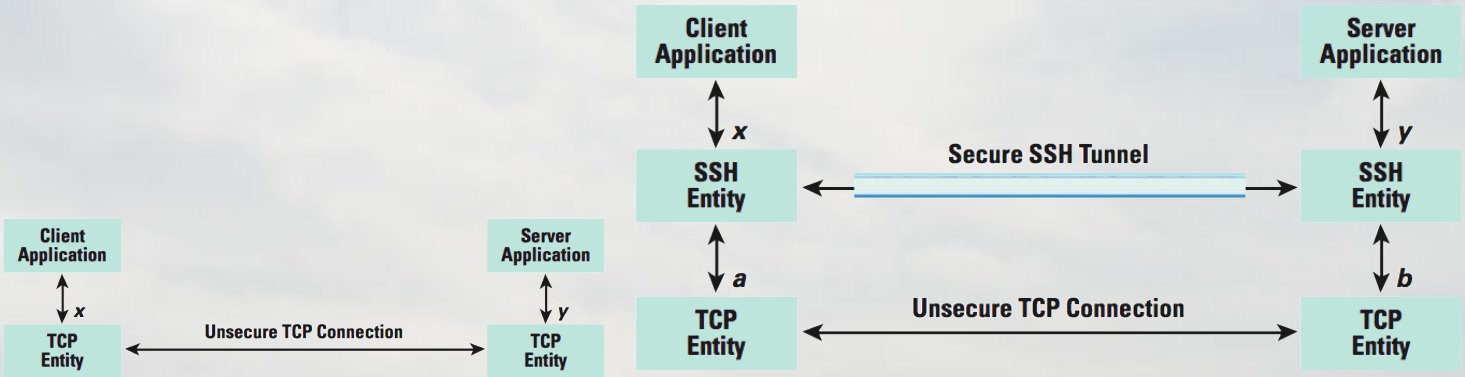
SP800-57-3_ch10



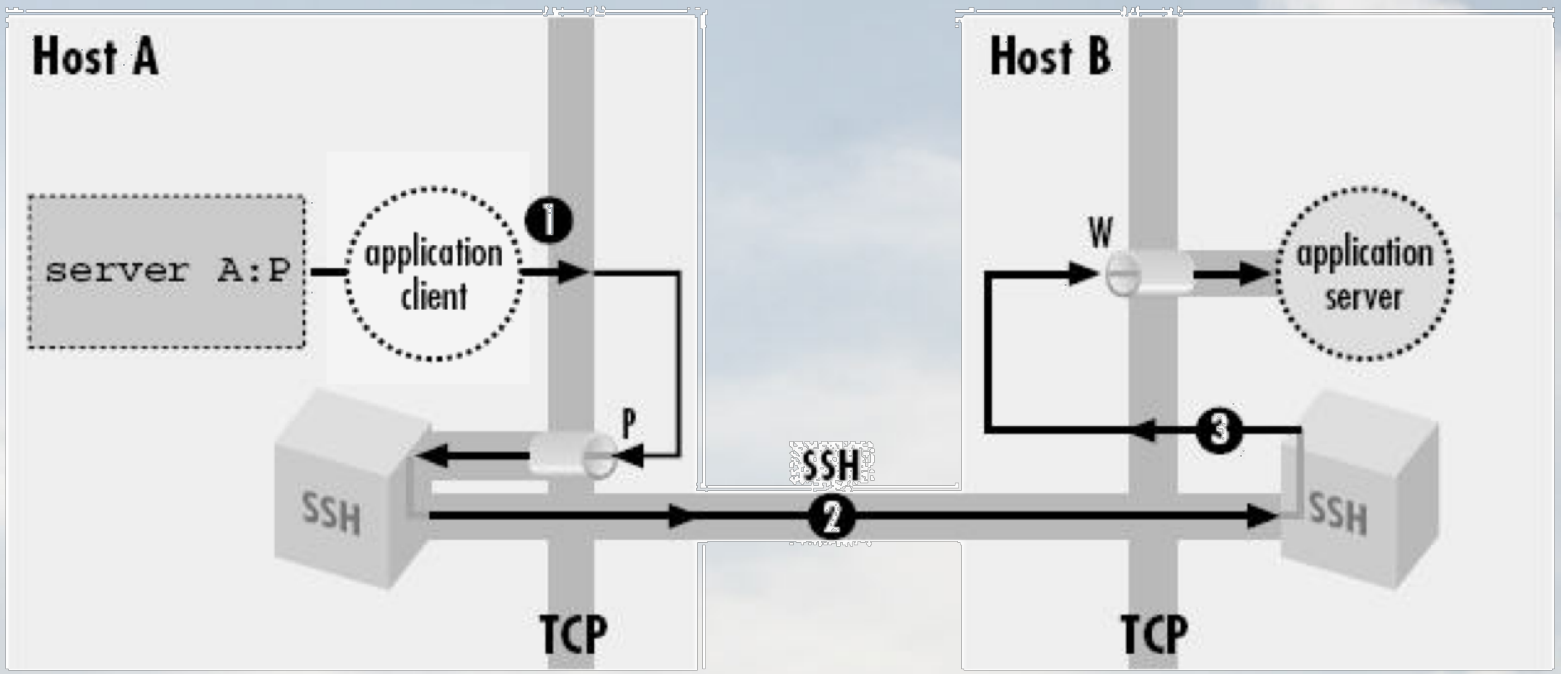
Reenvío de conexiones (*Port forwarding* or *SSH tunneling*)

Segurizar protocolos de aplicaciones reenviando su conexión TCP a través de una conexión SSH, cifrando de modo transparente de extremo a extremo.

- SSH establece un túnel seguro entre un puerto origen TCP del cliente (cualquiera) y el puerto TCP donde escucha el servidor.
  `ssh [-Nf] {-L,-R}`
  `[sourceHost:]fromPort:forwardToHost:onPort`
  `user@sshserver`



fuente: W. Stallings "SSH" IPJ-2009



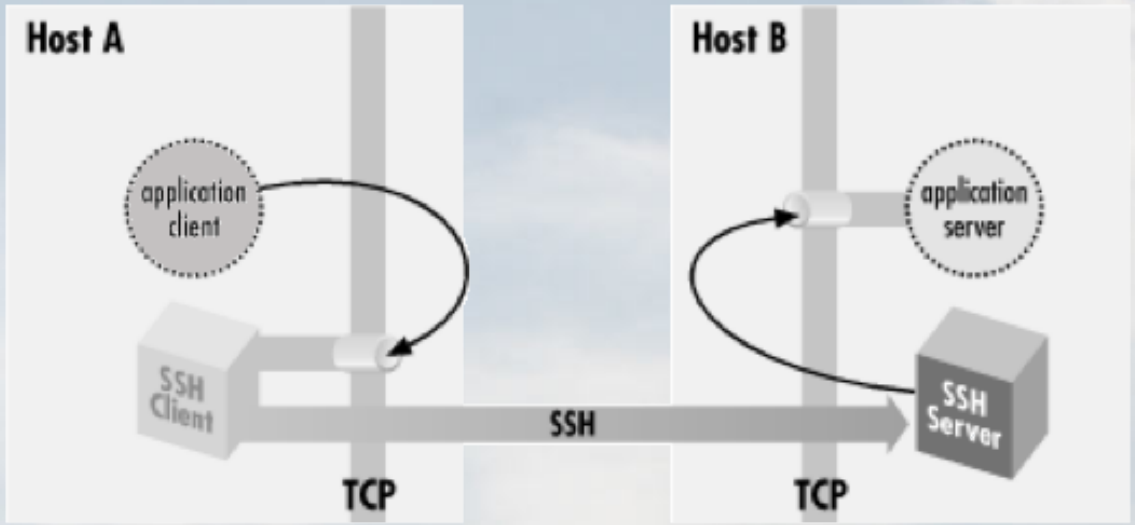

```
hostA$ ssh -L [localhost:]P:localhost:W user@HostB
```

fuente: D. Barret et al. "SSH, the secure shell", 2005

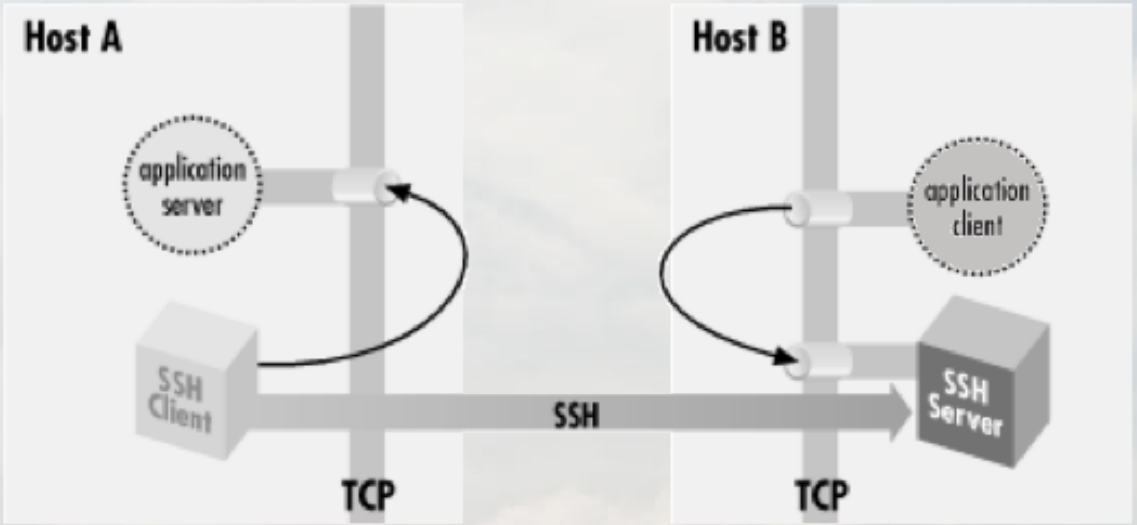


Local vs Remote port forwarding

Local



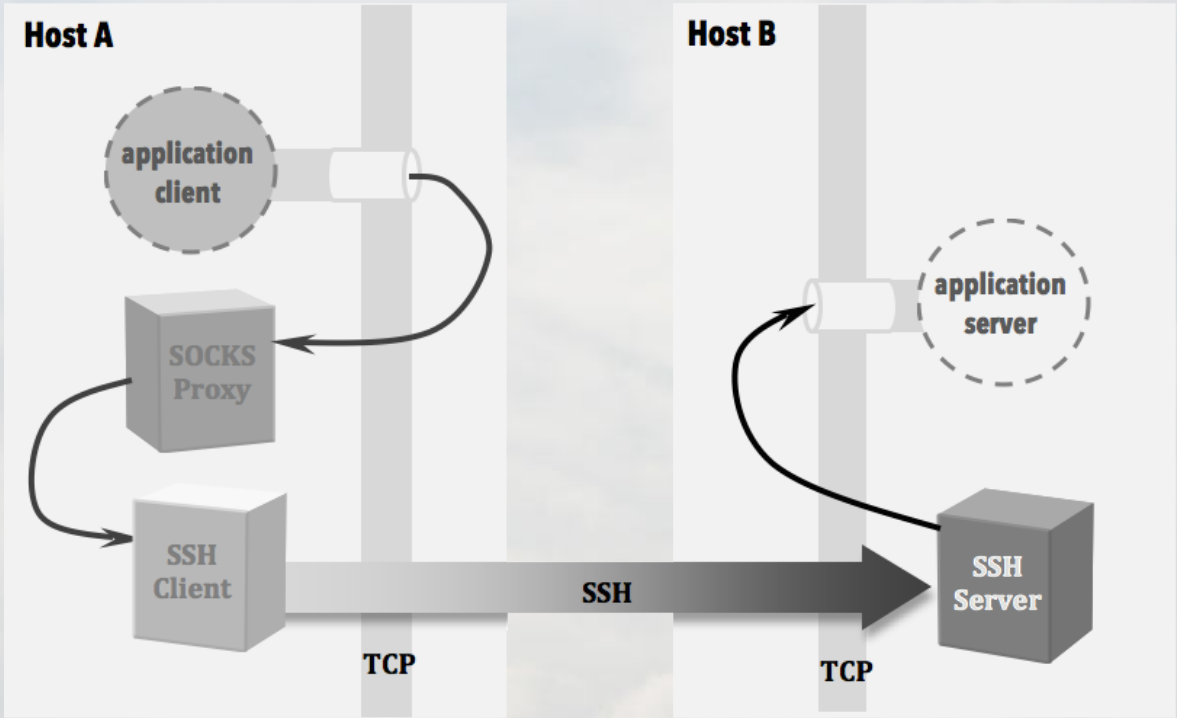
Remote



Dynamic Tunneling (SOCKS proxy)

Mediante SSH, convertir el host en un servidor *proxy SOCKS*.
Permite conexiones de clientes de distintas aplicaciones a través del cliente SSH local, que redirige a un servidor SSH, y este a su vez al servidor de la aplicación destino correspondiente.

`ssh -D` *`proxyport [user@]sshserver [-p sshport]`*



ACTIVIDAD: OpenSSH

Actividad: OpenSSH



E2EE protocols

- *End-to-end encrypted* (E2EE) *protocols* :
  - PGP
  - S/MIME



Pretty Good Privacy

- Desarrollado a principios de los 90 (por *Phil Zimmermann*). Combina técnicas de criptografía simétrica y asimétrica
- Proporciona los servicios de confidencialidad y autenticación del mensaje (integridad + autenticación del emisor) para aplicaciones de correo electrónico y almacenamiento de datos.
- Gestión de claves: creación, almacenamiento (en llaveros), firma de claves públicas (emisión certificados), verificación de firmas...
- Estándar internacional: OpenPGP (~~RFC 4880~~, RFC 9580), Post-Quantum Cryptography (RFC 9980). Ejemplo: SequoiaPGP (online)
- Estándar alternativo: LibrePGP Draft-RFC 4880bis; ejemplo: GnuPGP



PGP 2.6.3i (RFC 1991, *Informational*)

- Versión de PGP muy popular (fuentes disponibles, no controlada por instituciones gubernamentales o de estándares)
- Utiliza los algoritmos (*ciphersuite*):

Resumen	MD5
Cifrado asimétrico	RSA
Cifrado simétrico	IDEA
Compresión	ZIP
Compatibilidad SMTP	Base-64
Segmentación	-

- TAREA: Revisar cuáles son los algoritmos utilizados por la última versión de OpenPGP.



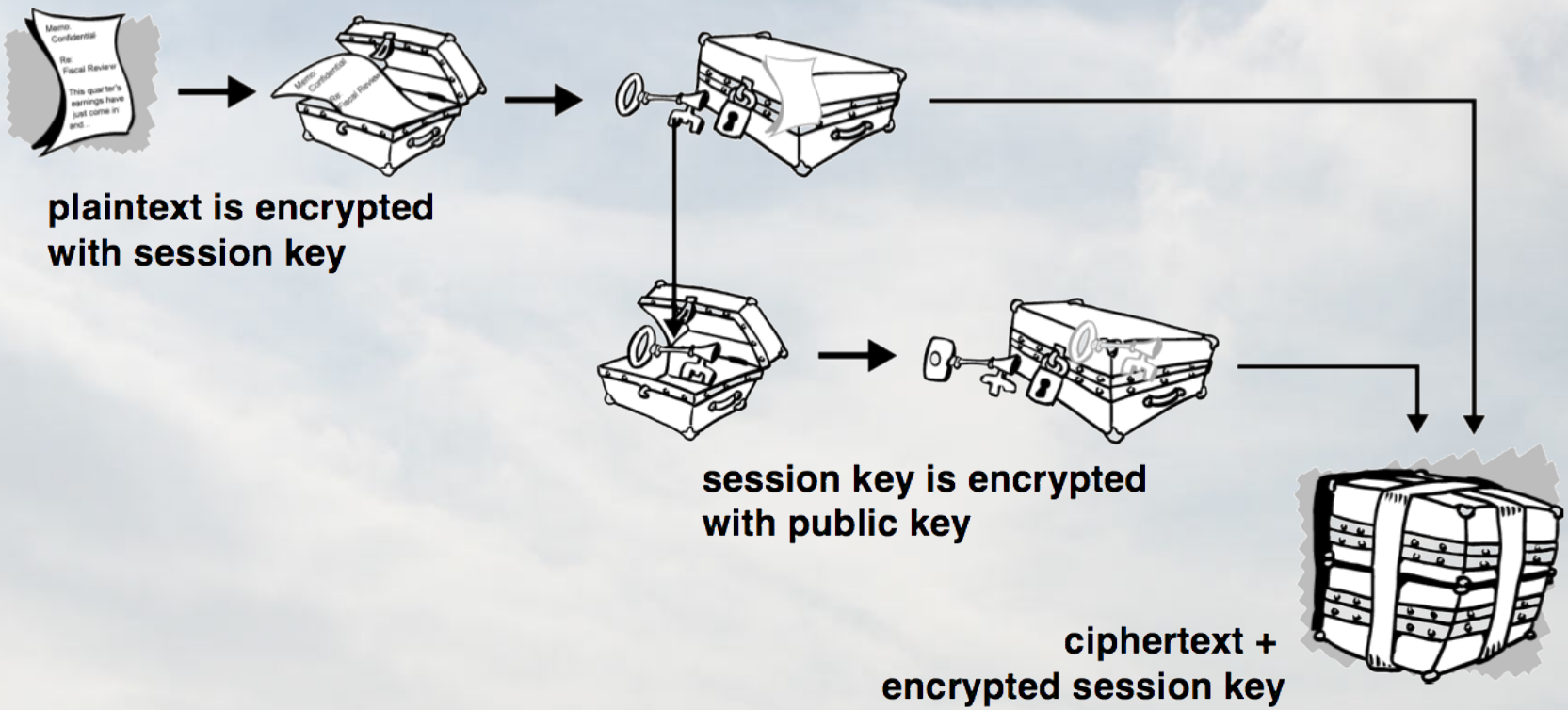
Observaciones

- El cifrado asimétrico se aplica únicamente a las claves simétricas (son cortas) y resúmenes.
- No se utiliza ninguna autoridad para la verificación de las claves públicas.
  PGP permite a un usuario firmar claves, y podremos confiar en la autenticidad de una clave si está firmada por un usuario de confianza (*web of trust*). Permite otorgar niveles de confianza a los usuarios.
- La información se comprime después de firmarla pero antes de cifrarla.
  Este posicionamiento no es arbitrario es mejor firmar un mensaje no comprimido
- PGP también realiza una segmentación para mensajes demasiado grandes



Confidencialidad

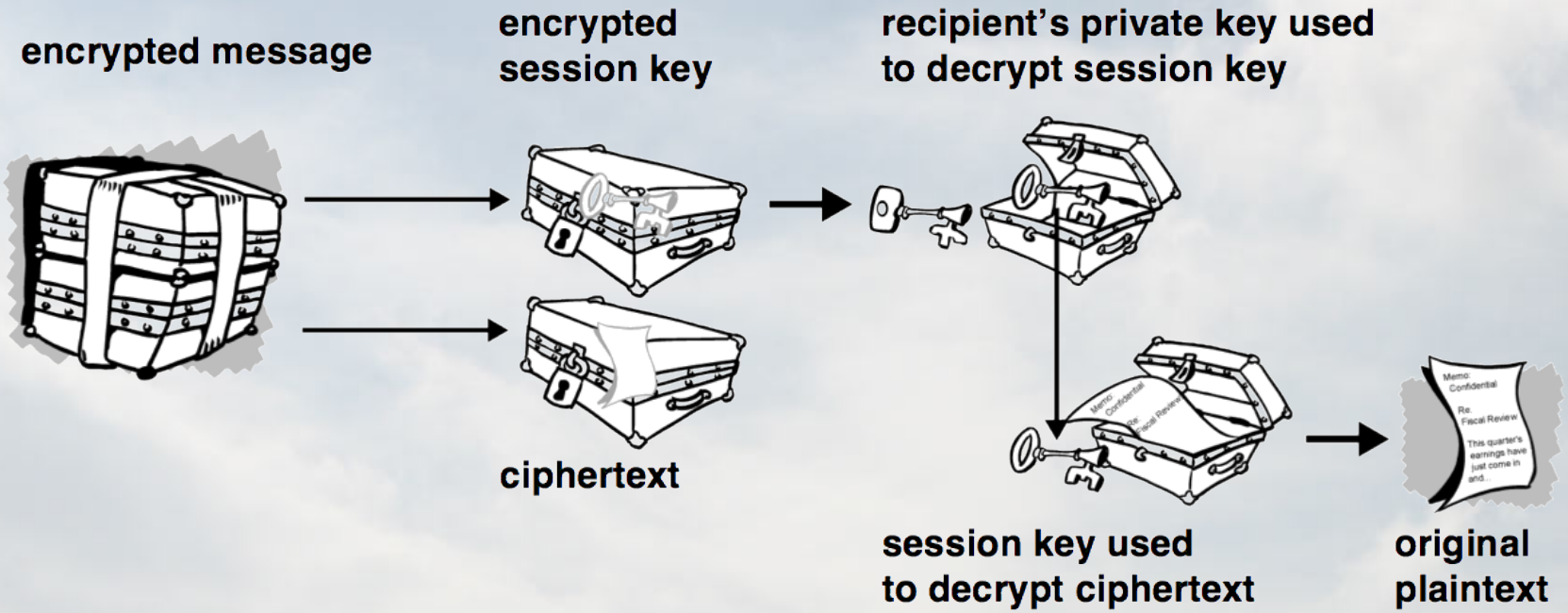
*Envío del mensaje (**sobre digital**)*



fuente:ftp.pgpi.org/pub/pgp/6.5/docs/english/IntroToCrypto.pdf

[https://www.cs.stonybrook.edu/sites/default/files/PGP70IntroToCrypto.pdf]

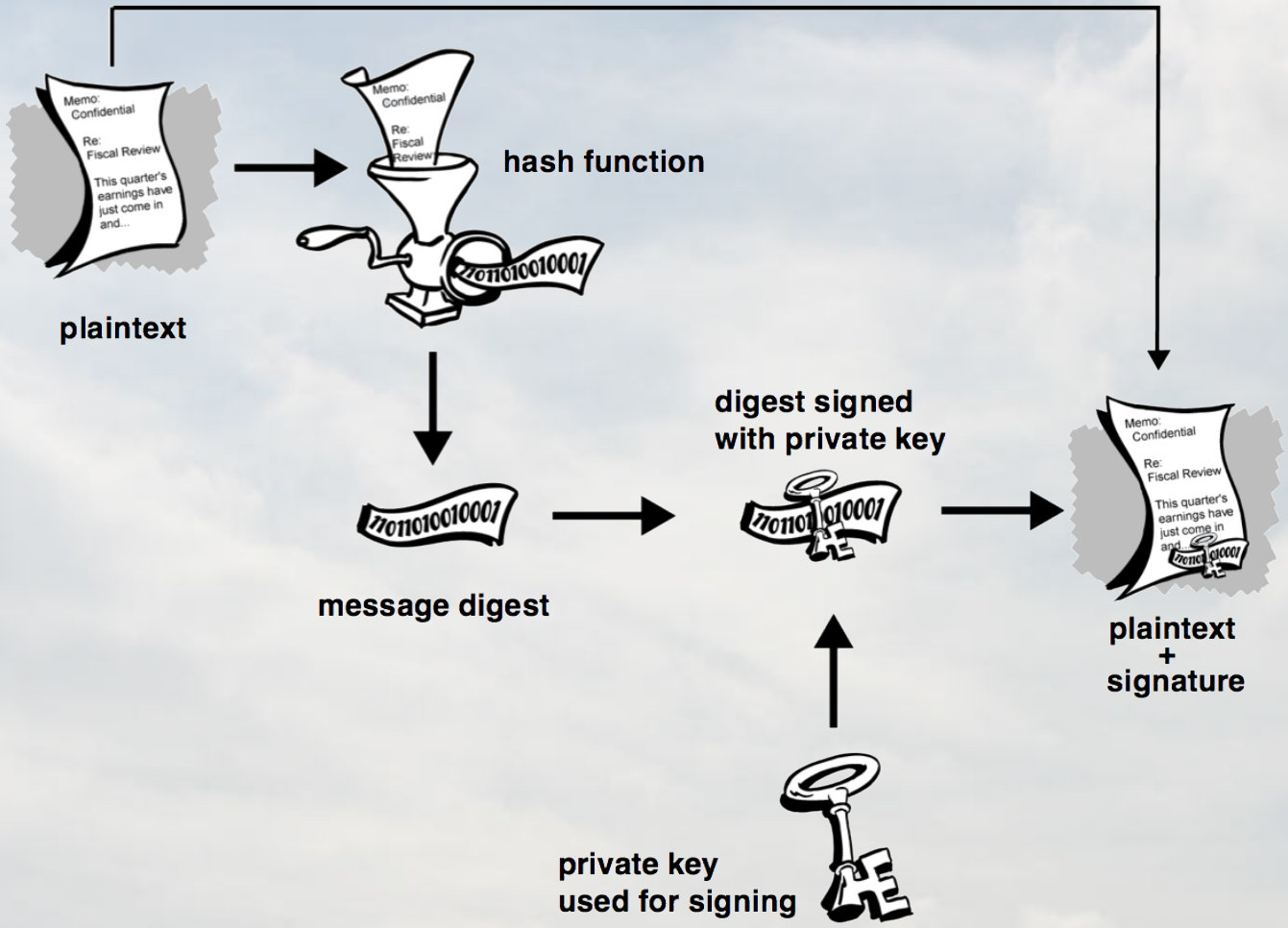
Recepción del mensaje



fuente:ftp.pgpi.org/pub/pgp/6.5/docs/english/IntroToCrypto.pdf

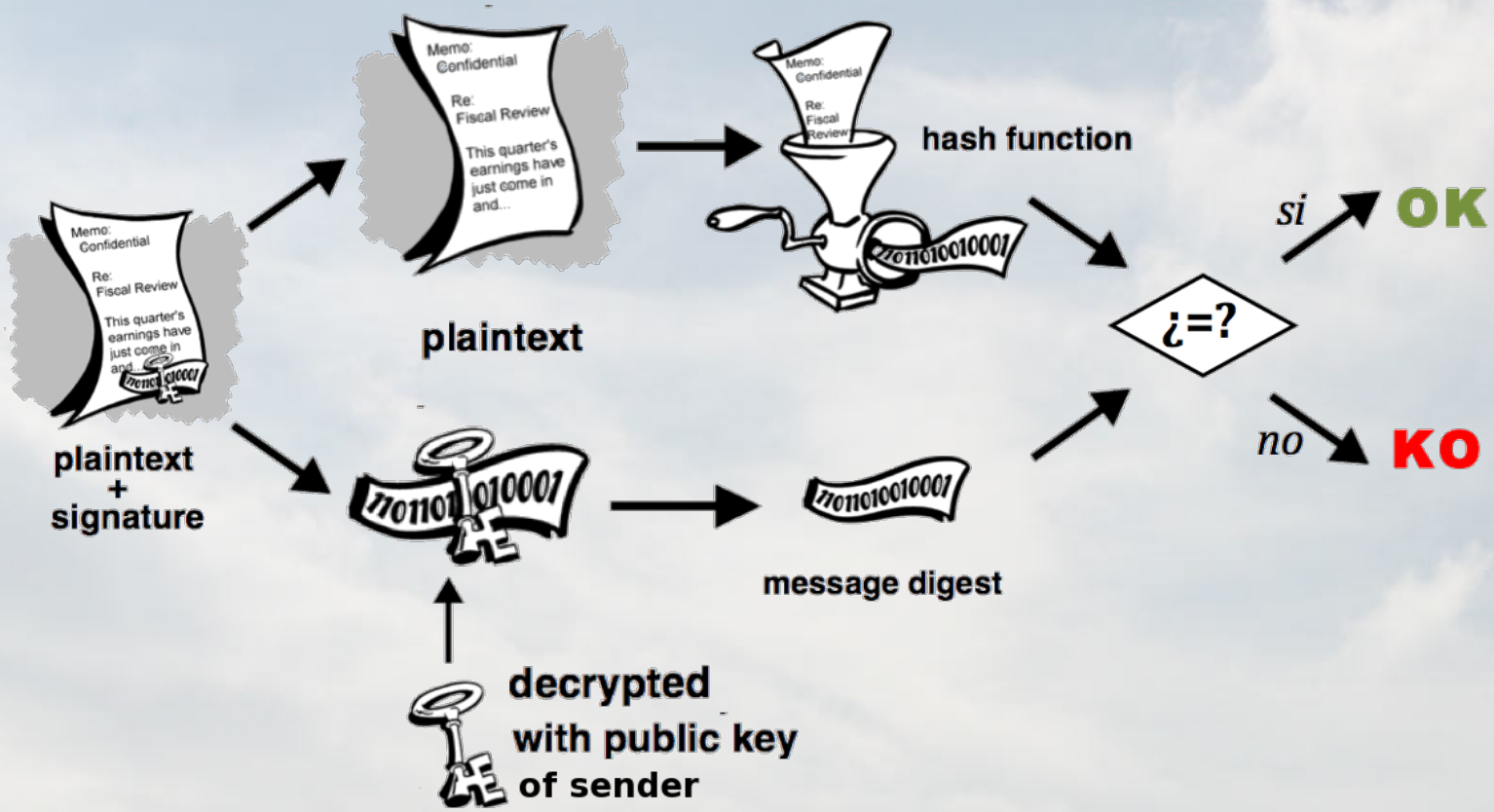
Autenticación del mensaje

Firma digital del mensaje



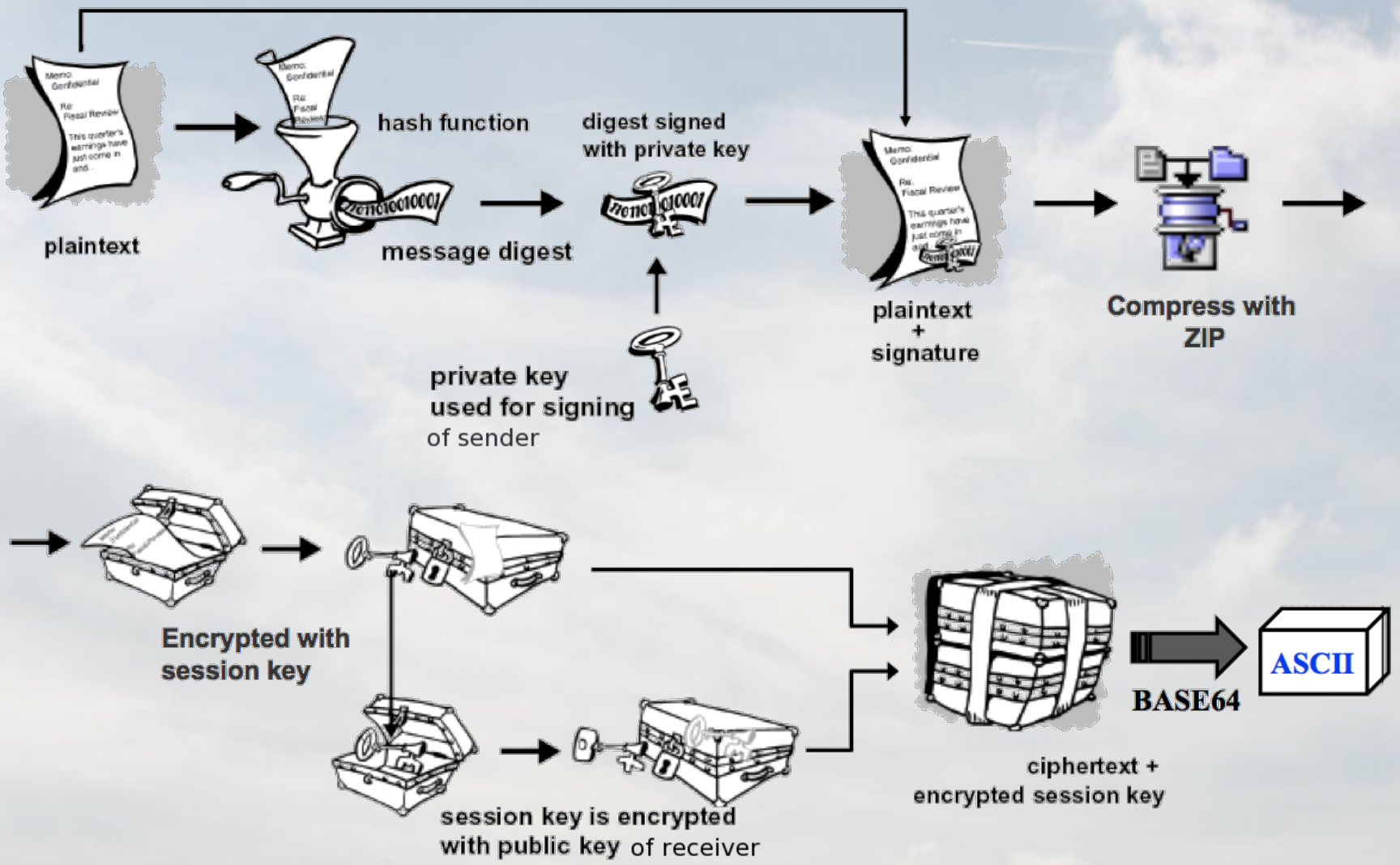
fuente:ftp.pgpi.org/pub/pgp/6.5/docs/english/IntroToCrypto.pdf

Verificación de la firma del mensaje



adaptación: ftp.pgpi.org/pub/pgp/6.5/docs/english/IntroToCrypto.pdf

Confidencialidad + Autenticación del mensaje +...



adaptación: ftp.pgpi.org/pub/pgp/6.5/docs/english/IntroToCrypto.pdf

Ejercicios

- Dibujar el esquema con los pasos a realizar por el receptor del mensaje correspondiente al envío del mensaje del esquema anterior.
- Dibujar el esquema correspondiente al envío de un mensaje a varios receptores.
- Con GnuPG realizar las siguientes pruebas:
  - Encriptar un fichero para múltiples receptores. Analizar el resultado (`gpg --list-packets`).
  - Desencriptar el fichero por uno de los receptores



Llaveros de claves

Llavero de claves privadas

Sello de tiempo	Identificador de clave	Clave pública	Clave privada cifrada	Identificador de usuario
:	:	:	:	:

S2K(String-to-key): convierte *passphrase* en una clave simétrica.

LLavero de claves públicas

Sello de tiempo	Identificador de clave	Clave pública	Confianza en el dueño	Identificador de usuario	Legitimidad de clave	Firma(s)	Confianza en la firma
:	:	:	:	:	:	:	:

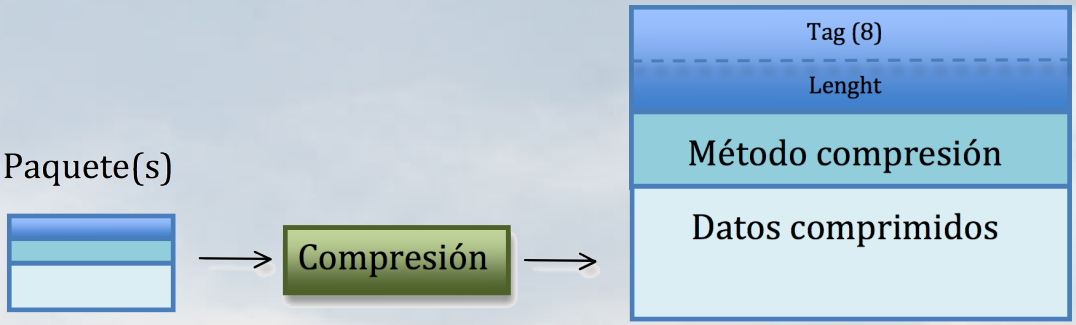


Formato general de los mensajes OpenPGP (9580)

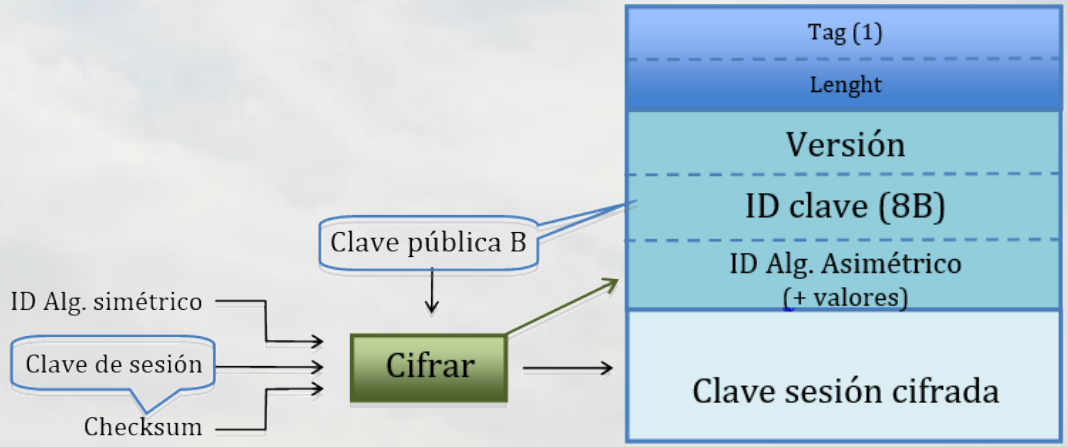
- Un **mensaje** OpenPGP está formado por una combinación secuencial o anidada de paquetes (componentes). Tipos de mensajes (RFC9580-10.3):
  - Cifrado, Firmado, Comprimido o Literal
- Cada **paquete** OpenPGP, consiste en una cabecera genérica que indica el tipo de paquete y su longitud, seguido por la cabecera (una serie de campos específicos según el tipo) y el cuerpo del paquete.
- Hasta 64 tipos de paquetes (RFC 9580-5):
  - (1) Clave de sesión cifrada con clave pública (PKESK)
  - (2) Firma (SIG)
  - (5) Clave de sesión (SECKEY)
  - (6) Clave pública (PUBKEY)
  - (8) Datos comprimidos (COMP)
  - (9) Datos cifrados con clave simétrica (SED)
  - (11) Datos literales (LIT)
  - (13) User ID (UID)
  - ...



Paquete de datos comprimidos (COMP):



Paquete de clave de sesión cifrada con clave pública (v3 PKESK):



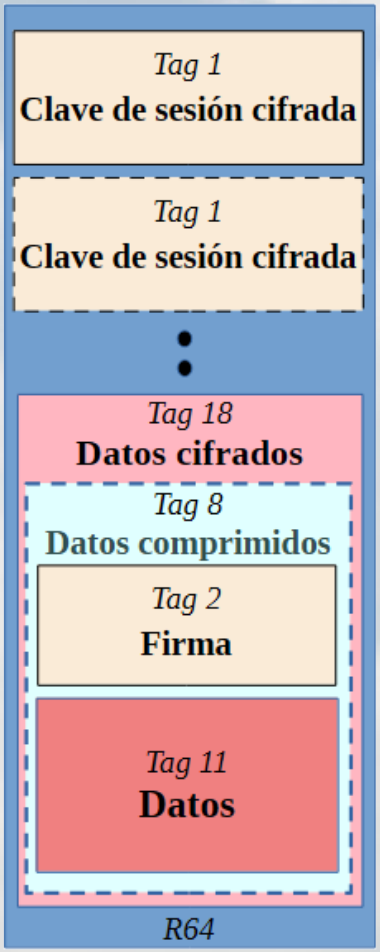
Mensaje: combinación de paquetes (RFC9580-10.3)



Cifrado



Firmado



Cifrado +Firmado

Ejemplos

Mensaje firmado

```
----BEGIN PGP SIGNED MESSAGE----
Hash: SHA2
Hola Ana:
La clase en el laboratorio E5 queda
suspendida hasta nuevo aviso.
Saludos,
Maite

----BEGIN PGP SIGNATURE----
Version: OpenPGP v2.0.76

9NWxUUOB57pv683i+e6gX42nRUoZUxUOSIRS
lyQtMifnicO8LLzMDF6Mfj7m1yYnnpoDcU4P
5vgEZJtlf1Vq31YAADviUq0=
=4LQP

----END PGP SIGNATURE----
```

Mensaje firmado y cifrado

```
----BEGIN PGP  MESSAGE----
Version: OpenPGP v2.0.76

LLzMDF6Mf683i+e6gX42nRUoZUxUOSIRmZxQpA
lJtaXJyYnnpoDgsDI0HoZ6cxbyzvrCiuDTtGTf
zdZ1lU3DJ44KSQ+1a9rvWgu9nJp9f9xcDGysTS
JKBi1MApuP3ZOb+op9f9NWxUUOB57pvSlyQtMi
fj7m1cUV09L4NVVp2UXf7z356ZuZ1vDqrfTZdu
P31dkFfELepLromYgsHTdWysIV5tYfeTF9wdIY
5y1zmrdf0HoZ6cxbyzvrCiuDTtGTfzdZ1lU3DJ
44KSQ+1a9rvWgu9nJp9f9NWxUUOB57pvSlyQtM
ifj7m1cU4P5vgEZJtlf1Vq31YAADviUq0==
=44KS

----END PGP MESSAGE----
```



Ejercicios

Ejercicios:

1. Genera un mensaje firmado ("`gpg --sign` *mensaje*"). Ejecuta el comando "`gpg --list-packets` *mensaje-firmado*" podras comprobar que la estructura de paquetes mostrada difiere de la presentada en el gráfico anterior. Indica las diferencias y explica porqué ha cambiado. Redibuja el nuevo gráfico para un mensaje firmado con la nueva estructura de paquetes.
2. ¿Qué diferencias existen entre los paquetes identificados con `tag=9`, `tag=18` y `tag=19`?. Dibujar el esquema correspondiente a estos paquetes.



Secure/Multipurpose Internet Mail Extension (RFC 8550-1,2634)

- Es una **mejora de seguridad** para el formato de correo electrónico de Internet MIME (extensión de RFC 822 para permitir procesar con SMTP información no-US-ASCII)
- Estándar IETF, con distinto objetivo
  - S/MIME –> estándar industrial para uso comercial e institucional
  - PGP –> seguridad del correo electrónico personal
- Proporciona autenticación, integridad de datos, confidencialidad y no repudio
- Gestión de claves: usa **certificados** X.509 firmados por CAs, el cliente de correo debe mantener un almacén de certificados de CAs raíz necesarios para verificar firmas.
- Versión 4 incorpora: Recibos firmados, etiquetas de seguridad, certificados de firma, listas de correo seguras



MIME (RFC 2045-47, 2077, 4289, 6838)

- Dota de estructura al cuerpo de los mensajes de correo electrónico. Cada sección tiene su propia cabecera.
- Define normas de **codificación** para mensajes no-US-ASCII.
- Incluye cinco campos extras en la cabecera que proporcionan información sobre el cuerpo del mensaje:
  - `MIME-Version`: Indica que se trata de un mensaje MIME y versión.
  - `Content-type`: tipo/subtipo; parámetros.
    Tipo: image, text, audio, video, application... Subtipo: jpeg, richtext, basic, mpeg, PostScript...
  - `Content-transfer-encoding`: Especifica como está codificado el mensaje: 7bit, binary, base64, x-token, radix64...
  - `Content-Id`: Identificación del mensaje (único).
  - `Content-Description`: String ASCII para el usuario, describe el contenido.

IANA-Content-types

Media Type Specifications and Registration Procedures



Ejemplo simplificado

```
From: gorka@boom.org
To: ane@ehu.edu
Subject: Mi foto
MIME-Version: 1.0
Content-Type: multipart/mixed; Boundary="StartOfNextPart"

--StartOfNextPart
Content-Type: text/plain; charset=ISO-8859-1; format=flowed
Content-Transfer-Encoding: 8bit
Ahí te va mi foto

--StartOfNextPart
Content-Transfer-Encoding: base64
Content-Type: image/jpeg
base64 encoded data .....
.........................
......base64 encoded data

--StartOfNextPart
Content-type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7BIT
Espero que no tengas problemas para verla.
•
```

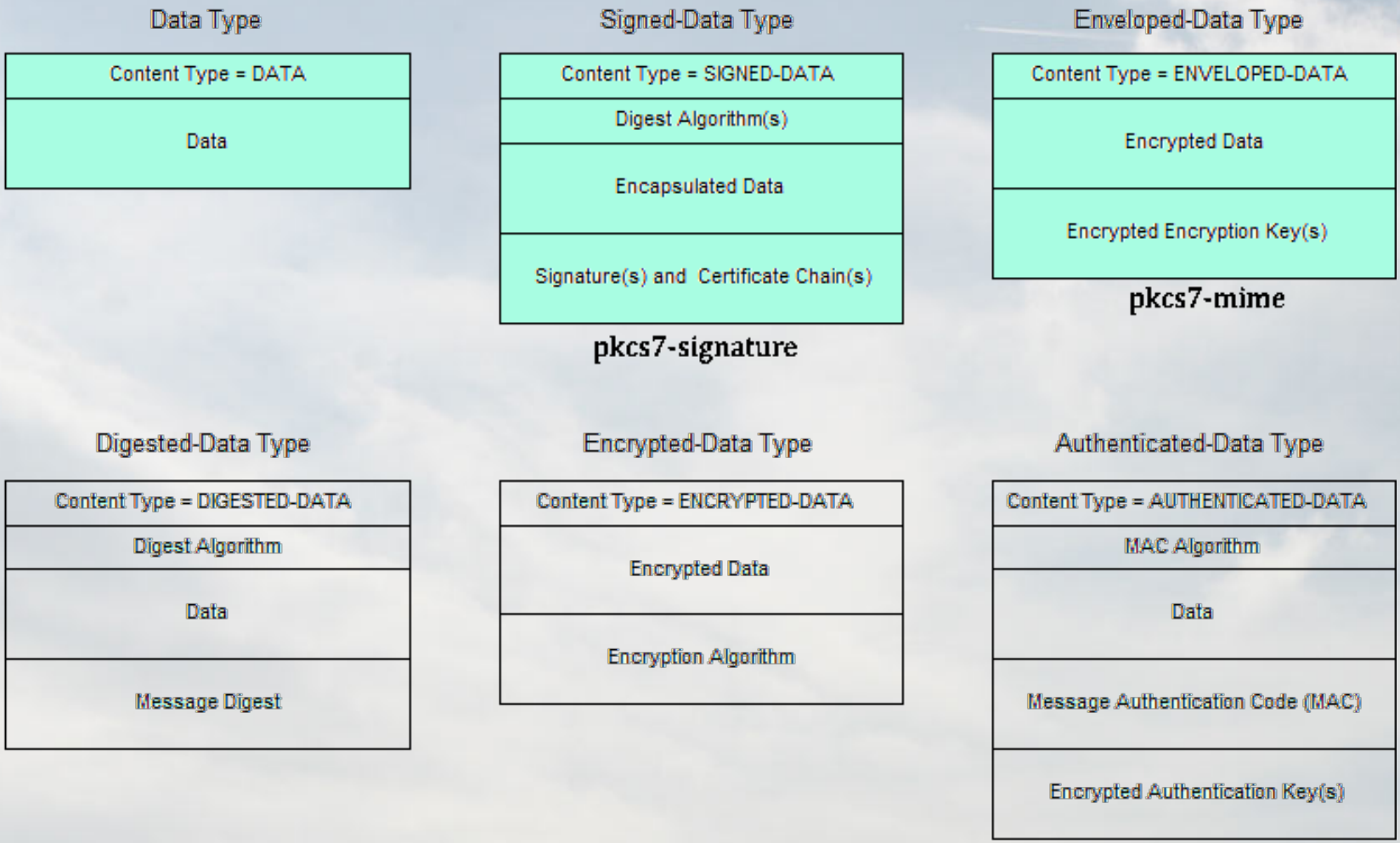


S/MIME (RFC 8551- v 4.0)

- Incorpora **servicios de seguridad** a MIME, permitiendo nuevos tipos de contenidos
- Se apoya en la especificación CMS (Cryptographic Message Syntax - **RFC 5652**) para encapsular y proteger el contenido multimedia MIME.
- Los tipos de contenido se pueden anidar arbitrariamente para proporcionar múltiples niveles de protección.



Tipos CMS



fuente: www.vocal.com/secure-communication/cryptographic-message-syntax-cms/

Combinaciones en un mensaje S/MIME (datos MIME y tipos CMS):

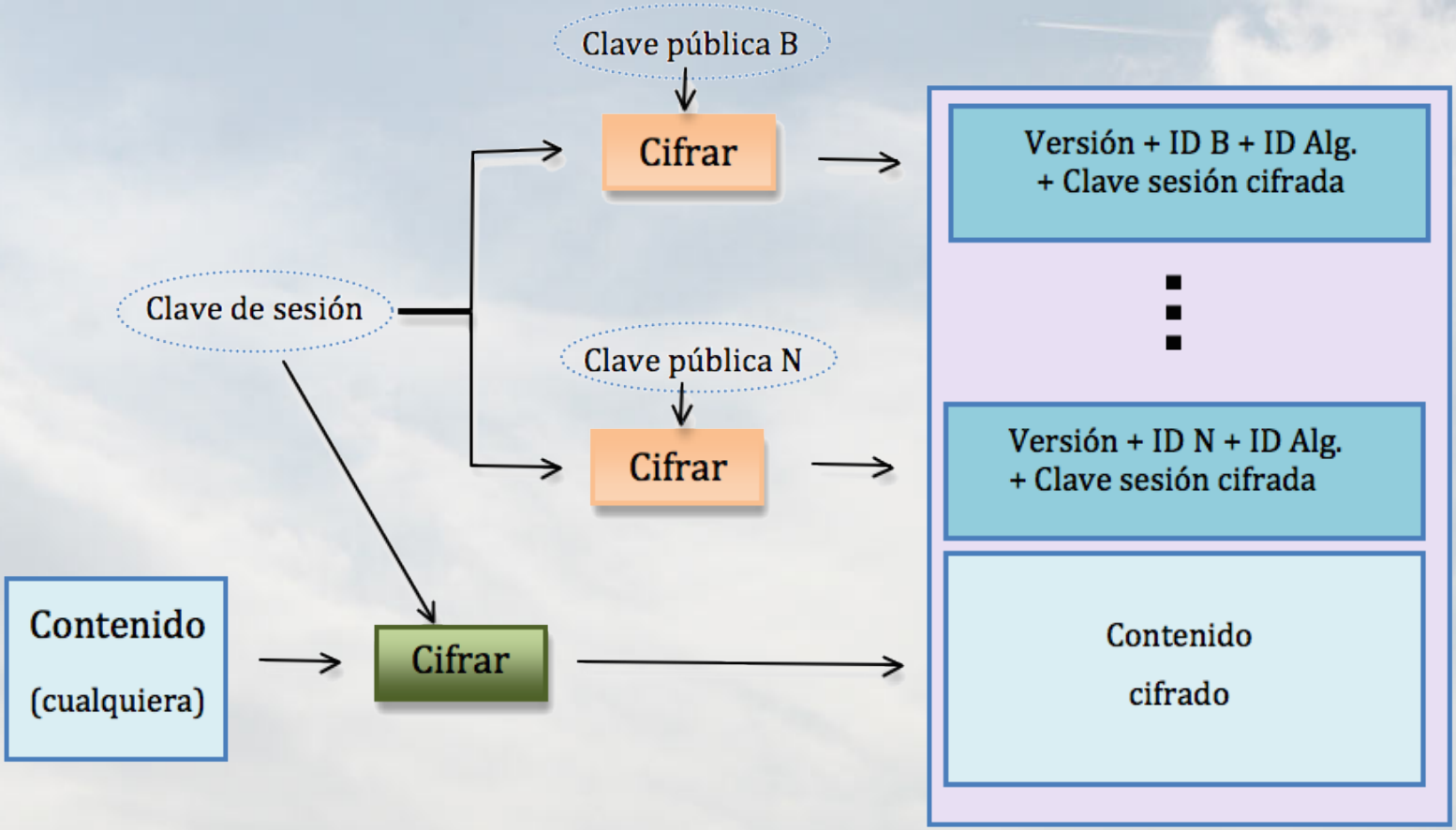
Mensaje firmado	*Signed-Data[Data MIME]*
	`content-type: multipart/signed;`
	`protocol="application/pkcs7-signature";`

Mensaje cifrado	*Enveloped-Data[Data MIME]*
	`content-type: application/pkcs7-mime;`
	`smime-type=enveloped-data;`

Mensaje firmado y cifrado:	*Enveloped-Data[Signed-Data [Data MIME]]*
	`content-type: application/pkcs7-mime;`
	`smime-type=enveloped-data;`



Enveloped-data content type (RFC 5652)



ID *x* : *certificado o clave de x*

Ejemplos

```
Date: Mon, 1 Mar 2004 11:46:10 +0100
From: usuario-1@uoc.edu
Subject: Ejemplo 1
To: usuario-2@uoc.edu
MIME-Version: 1.0
Content-Type: application/pkcs7-mime; smime-type=enveloped-data;
        name="smime.p7m"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="smime.p7m"

MIAGCSqGSIb3DQEHA6CAMIACAQAxgDBzAgEAMC8wKjELMAkGA1UEBhMCRVMxDDAKBgNVBAoT
A1VPQzENMAsGA1UEAxMEQ0EtMQIBBjANBgkqhkiG9w0BAQEFAAQuCYHs970aZmmqKTr3gemZ
LzHVtB266O/TrIv4shSvos8Ko8mUSQGov0JSIugmeDBzAgEAMC8wKjELMAkGA1UEBhMCRVMx
DDAKBgNVBAoTA1VPQzENMAsGA1UEAxMEQ0EtMQIBBzANBgkqhkiG9w0BAQEFAAQuC14oIhps
+mh8Wxp79A81uv2ltG3vt6J9UdJQcrDL92wD/jpw1IKpoR224LT4PQAAMIAGCSqGSIb3DQEH
ATARBgUrDgMCBwQIZbTj6XqCRkGggARAF8K8apgPtK7JPS1OaxfHMDXYTdEG92QXfAdTPetA
FGuPfxpJrQwX2omWuodVxP7PnWT2N5KwE1oc6faJY/zG0AAAAAAAAAAAAA=
```

© FUOC • P06/M2107/01772



```
Date: Mon, 1 Mar 2004 11:47:40 +0100
From: usuario-1@uoc.edu
Subject: Ejemplo 3
To: usuario-2@uoc.edu
MIME-Version: 1.0
Content-Type: multipart/signed; boundary="20040301104740";
        protocol="application/pkcs7-signature"; micalg=md5

--20040301104740
Content-Type: text/plain

Ejemplo de mensaje firmado.

--20040301104740
Content-Type: application/pkcs7-signature; name="smime.p7s"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="smime.p7s"

MIAGCSqGSIb3DQEHAqCAMIACAQExDjAMBggqhkiG9w0CBQUAMIAGCSqGSIb3DQEHAQAAoIAw
ggFIMIIBBaADAgECAgEGMA0GCSqGSIb3DQEBBAUAMCoxCzAJBgNVBAYTAkVTMQwwCgYDVQQK
EwNVT0MxDTALBgNVBAMTBENBLTEwHhcNMDAwOTAxMDAwMDAwWhcNMTAwOTAxMDAwMDAwWjBV
MQswCQYDVQQGEwJFUzEMMAoGA1UEChMDVU9DMSUwIwYJKoZIhvcNAQkBFhZ1c3VhcmktMUBj
YW1wdXMudW9jLmVzMREwDwYDVQQDEwh1c3VhcmktMTBJMA0GCSqGSIb3DQEBAQUAAzgAMDUC
Lgw1Es5ojfSWtFCPLLOdONBOz+8u96IVp1GIYqVU2ewWQbH8TAd0UPLvPgKlu2cCAwEAATAN
BgkqhkiG9w0BAQQFAAMuAAJFmsOwv8wm/1Vl1vE0yluMpzDAU89Ub0tOo5nyxN0AXT1ScWxK
PmMTQAEA8QAAMYAwgYECAQEwLzAqMQswCQYDVQQGEwJFUzEMMAoGA1UEChMDVU9DMQ0wCwYD
VQQDEwRDQS0xAgEGMAwGCCqGSIb3DQIFBQAwDQYJKoZIhvcNAQEBBQAELgMzAj7h+9GoE+EW
P/AHuAj46vudJh+So/V/FuN1OrC70HmOyfEMiz49RixuhAgAAAAAAAAAA==
--20040301104740--
```

© FUOC • P06/M2107/01772

RFC 3156 "MIME Security with OpenPGP"

En muchas situaciones es ventajoso combinar PGP con MIME.

RFC 3156 especifica 3 content-types:

- `application/pgp-encrypted`
- `application/pgp-signature`
- `application/pgp-keys`

Mensaje firmado y cifrado



fuente: https://www.esecurity.ch/E2EEMessaging.pdf